Restaurant Scheduler - Updated CORE with Edge Cases

CORE: Build the Schedule

Check the rules: Availability, roles, coverage every hour, at most 8 hours, one shift per person.

- If no valid schedule exists: *do not create an invalid schedule. Tell the manager which requirement could not be satisfied, such as no available employee or an uncovered required role.*
- If an employee would be assigned overlapping shifts: *reject the assignment and search for another valid assignment.*
- If there are not enough qualified employees for a required role: *tell the manager which role and hours cannot be covered.*

Breaks: Any shift over 6 hours gets a break, and the role stays covered.

- If a required break would leave the employee's role uncovered: *try another valid break time.*
- If no break time can satisfy both the break requirement and role coverage: *report the conflict to the manager instead of creating an invalid schedule.*

Pick the best schedule: Compare valid schedules using the manager's goals, then post the best one.

- If several valid schedules exist: compare them using the manager's defined goals in priority order.
- If the scheduling method cannot determine a better solution: do not violate any hard constraints just to improve the schedule.

CORE: Handle a Call-Out

Fix the schedule: First try one replacement. If nobody can cover alone, find the smallest set of changes. If nothing works, say which role and hours can't be covered.

- If one qualified and available employee can cover the missing shift: *use that as the first repair option.*
- If no single employee can cover the entire missing shift: *find the smallest set of schedule changes that restores coverage.*
- If a possible replacement creates another violation: *reject that repair and try another solution.*
- If the repair causes a break or role-coverage conflict: *re-check the affected schedule and find another valid repair.*
- If no valid repair exists: *tell the manager exactly which role and hours cannot be covered.*

LATER

Multiple simultaneous call-outs:

- *If several employees call out at the same time, the system may need to repair several parts of the schedule simultaneously. This can be addressed after the single-call-out workflow works correctly.*

Large-scale scheduling:

- *Investigate how well the scheduling method performs when the number of employees, roles, and constraints becomes much larger.*